Workshop Sheet 03: Operators and Expressions

Module: CMP1138: Foundations of Programming
Week: 3
Date: 13 October 2026

Objectives

- Use arithmetic, assignment, compound-assignment, increment/decrement, and modulus operators.
- Use comparison and logical operators to form Boolean expressions.
- Predict and control the order in which expressions are evaluated.
- Apply operators in short, practical console programs.
- Write clear code using meaningful variables, parentheses, and formatted output.

Prerequisites

- Week 2 knowledge of variables, data types, console input, and formatted output.
- A new Console App project for this workshop. Create it with `.NET: New Project...`, or run `dotnet new console` in an empty `Workshop3` folder.
- Familiarity with `Console.ReadLine()`, `Console.WriteLine()`, parsing, and string interpolation.

Tip: Use `decimal` values for money and add the `m` suffix to decimal literals, for example `0.20m`.

Task 1: Arithmetic Playground

Objective

To practise arithmetic operators and see the effect of integer division.

Instructions

1. Ask the user for two whole numbers named `a` and `b`.
2. Calculate and display their sum, difference, product, quotient, remainder, and average.
3. Use interpolation to display clearly labelled results.
4. Calculate the average with real-number division so that fractions are not discarded.
5. Change one expression by adding parentheses, then explain in a code comment how the result changes.

Expected Outcome

```
Enter first number: 7
Enter second number: 3

Sum: 10
Difference: 4
Product: 21
Quotient: 2
Remainder: 1
Average: 5.00
```

Task 2: VAT and Final Price Calculator

Objective

To use `decimal` values and arithmetic expressions for money.

Instructions

1. Ask the user for an item price and a VAT rate as a percentage.
2. Store both inputs as `decimal` values.
3. Calculate the VAT amount using `price * rate / 100m`.
4. Calculate the final price by adding VAT to the item price.
5. Print both amounts to two decimal places with pound signs.

Expected Outcome

```
Item price (£): 25.00
VAT rate (%): 20
VAT amount: £5.00
Final price: £30.00
```

Task 3: Membership Eligibility Checker

Objective

To form Boolean expressions using comparison and logical operators.

Scenario

A community gym needs to check whether a member qualifies for standard access, premium access, and independent use of the gym.

Instructions

1. Ask the user for their age and their membership duration in years.
2. Create a Boolean variable for standard access: age from 18 to 65 inclusive.
3. Create a Boolean variable for premium access: over 60 years old *or* a member for more than five years.
4. Create a Boolean variable that reports whether the member is not underage.
5. Print the three Boolean results with descriptive labels.

Expected Outcome

```
=== Gym Membership Eligibility Test ===
Enter age: 25
Enter membership years: 3

Standard access (18-65): True
Premium access (senior OR loyal): False
Not underage: True
```

Task 4: Monthly Salary Breakdown

Objective

To combine arithmetic, assignment, and formatted money output.

Scenario

A small company is prototyping a payroll calculator. It needs to add a percentage bonus and deduct tax and pension contributions from a monthly salary.

Instructions

1. Ask for base salary, bonus rate, tax deduction, and pension contribution. Store money values as `decimal`.
2. Calculate `bonus = baseSalary * bonusRate / 100m`.
3. Calculate gross salary and then net salary.
4. Use `+=` or `-=` at least once to show a compound assignment in a suitable calculation.
5. Print a clearly aligned breakdown, formatting every money value to two decimal places.

Expected Outcome

```
=== Payroll Calculator ===
Enter base salary (£): 2500
Enter bonus rate (%): 10
Enter tax deduction (£): 300
Enter pension contribution (£): 150
```

```
Base salary          : £2500.00
Bonus (10%)          : £250.00
Gross salary         : £2750.00
Tax + pension        : £450.00
Net salary           : £2300.00
```

Task 5: Student Performance Evaluator

Objective

To calculate an average and classify it with Boolean expressions.

Instructions

1. Ask the user for three module marks from 0 to 100.
2. Calculate the average using real-number division: `(mark1 + mark2 + mark3) / 3.0`.
3. Create Boolean variables for distinction (at least 70), pass (at least 40 but below 70), and fail (below 40).
4. Print the average to two decimal places and display each Boolean result.
5. Test the program with values that produce one result in each category.

Expected Outcome

```
=== Student Performance Evaluator ===
Enter mark 1: 80
Enter mark 2: 65
Enter mark 3: 70

Average: 71.67
Distinction: True
Pass: False
Fail: False
```

Task 6: Cinema Kiosk Bug Hunt

Objective

To diagnose common operator, precedence, type, and formatting mistakes.

Instructions

1. Copy the following program exactly. It contains several errors and misleading calculations.
2. Use the Error List, compiler messages, and the Week 3 slides to identify the issues.
3. Correct the program so it uses `decimal` consistently for money, calculates voucher rewards correctly, and formats money to two decimal places.
4. Add a short comment next to each correction explaining the original problem.

```
Console.WriteLine("=== Cinema Kiosk ===");

Console.Write("Ticket price (£): ");
double ticketPrice = double.Parse(Console.ReadLine()!);
Console.Write("Number of tickets: ");
int ticketCount = int.Parse(Console.ReadLine()!);
Console.Write("Snack price (£): ");
double snackPrice = double.Parse(Console.ReadLine()!);
Console.Write("Number of snacks: ");
int snackCount = int.Parse(Console.ReadLine()!);
Console.Write("Voucher tokens: ");
int tokens = int.Parse(Console.ReadLine()!);

double ticketSubtotal = ticketPrice * ticketCount;
double snackSubtotal = snackPrice * snackCount;
Console.WriteLine("Ticket subtotal: £" + ticketSubtotal + snackSubtotal);

int freeSnacks = tokens % 3;
tokens = tokens / 3;
int leftoverTokens = tokens % 3;

double vatRate = 20 / 100;
double vat = (ticketSubtotal + snackSubtotal) * vatRate;
decimal totalDue = (decimal)(ticketSubtotal + snackSubtotal) + (decimal)vat;

Console.WriteLine("\n--- Bill ---");
Console.WriteLine("Free snacks from tokens: " + freeSnacks + ", leftover: " +
↪  leftoverTokens);
Console.WriteLine("VAT (20%): £" + vat);
Console.WriteLine("TOTAL: £" + totalDue);
```

Expected Outcome

A clean, compiled program that prints the ticket and snack subtotals, free-snack count, leftover tokens, VAT, and total due. Test it with ticket price £8.50, three tickets, snack price £3.00, two snacks, and four tokens.

Task 7: Expression Prediction Table

Objective

To reason about type, precedence, and Boolean results before running code.

Instructions

1. Predict the output of each expression below before adding it to a program.
2. Run the expressions, compare your predictions with the actual output, and correct any differences.
3. For every expression that surprises you, add parentheses or change a numeric literal to explain the behaviour.

```
Console.WriteLine(10 / 4);
```

```
Console.WriteLine(10 / 4.0);
Console.WriteLine("Total: " + 10 + 4);
Console.WriteLine("Total: " + (10 + 4));
Console.WriteLine(3 + 4 * 2);
Console.WriteLine((3 + 4) * 2);
Console.WriteLine(18 % 5 == 3 && 7 > 2);
```

Stretch Task: Vending Machine Change System

Objective

To use `decimal`, integer division, and modulus to produce a useful change breakdown.

Instructions

1. Ask for a product price and amount paid as `decimal` values. Assume the user pays enough.
2. Calculate the change due, then convert it to whole pence: `int pence = (int)(change * 100m)`.
3. Starting with the largest coin, calculate the number of £2, £1, 50p, 20p, 10p, 5p, 2p, and 1p coins needed.
4. After each coin, use `%` to keep only the pence still unallocated.
5. Print the change due with two decimal places followed by a neat list of coin counts.

Example

```
Product price: £1.30
Money paid: £2.00

Change due: £0.70
50p coins: 1
20p coins: 1
```